

Chapter 05 암호화 해시 함수 및 기타

목차

5.1 개요

5.2 암호화 해시 함수란?

5.3 생일 문제

5.4 생일 공격

5.5 비암호화 해시

5.6 SHA-3

5.7 HMAC

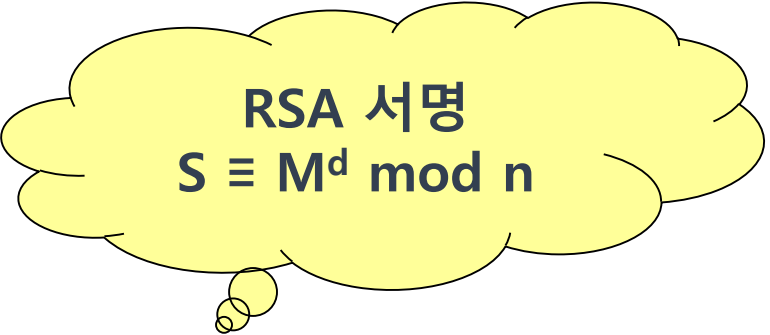
5.8 암호 해시 응용 프로그램

5.9 기타 암호 관련 주제

5.10 요약

개요

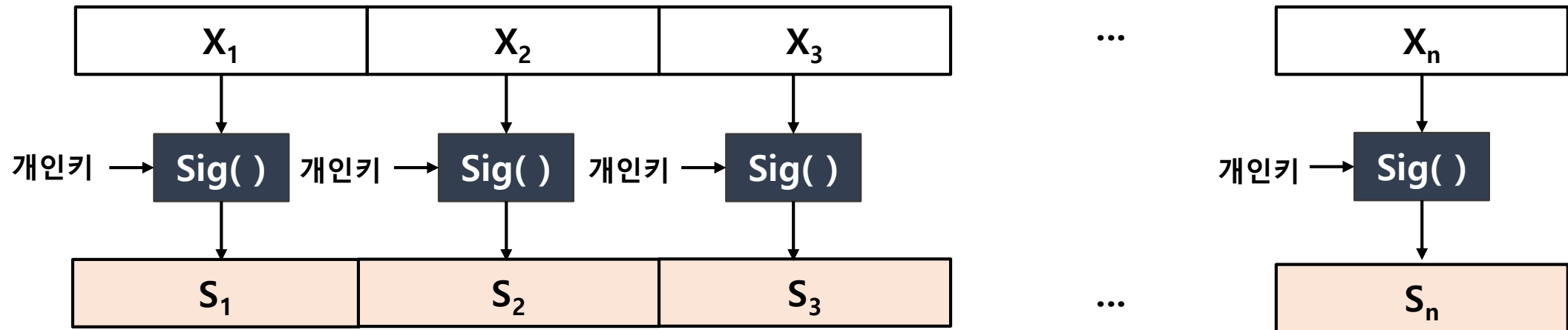
- 현대 암호 기술의 많은 응용 분야에서 해시 함수가 사용
- 특히, 전자서명에 매우 중요한 역할을 함
 - RSA 서명의 경우, 모듈러 n 은 1024bit에서 3072bit 범위에서 사용
 - 한번에 서명할 수 있는 메시지의 크기를 모듈러 n 의 크기로 제한됨
- 그렇다면, 긴 문서의 서명은 어떻게 생성할 것인가?



RSA 서명
 $S \equiv M^d \pmod{n}$

개요

- 직관적인 해결책 : 메시지를 모듈러 n 의 길이로 나누어 각 블록에 대해 서명



- 메시지 $X = X_1 || X_2 || \dots || X_n$ ($|X_i| = n$)
- 서명 $S = S_1 || S_2 || \dots || S_n$ ($|S_i| = n$)

➤ 문제점 1 : 높은 계산적 부담

- 전자서명은 큰 정수의 모듈러 지수승과 많은 많은 양의 연산이 필요한데, 이를 여러 번 수행할 경우 많은 양의 계산이 필요함

➤ 문제점 2 : 메시지 오버헤드

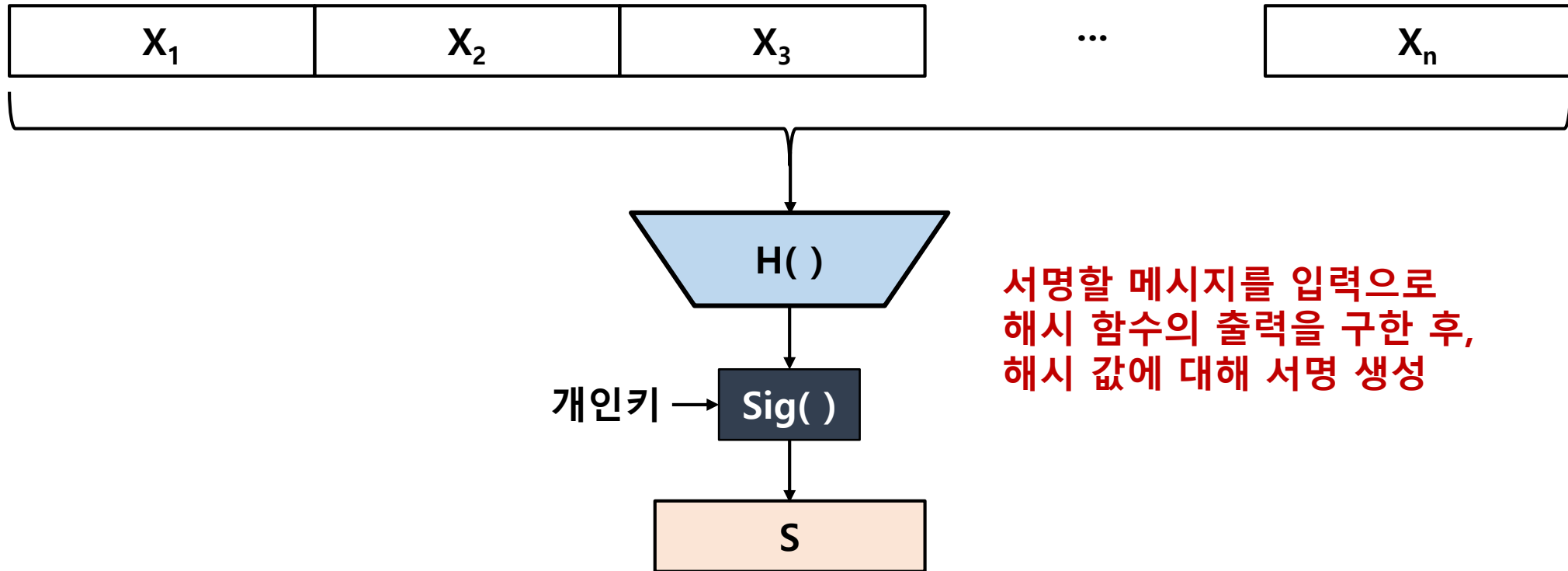
- 서명은 메시지와 함께 전송되어야 하는데, 이 경우 메시지의 길이는 2배로 늘어남

➤ 문제점 3 : 안전성 문제

- 각 메시지 블록에 대응되는 서명을 삭제하거나, 순서를 재 배열하는 등의 공격 가능

➤ 해결책

- 임의의 길이를 갖는 긴 메시지에 대해 **지문**의 역할을 하는 짧은 메시지를 생성해주는 해시 함수 이용



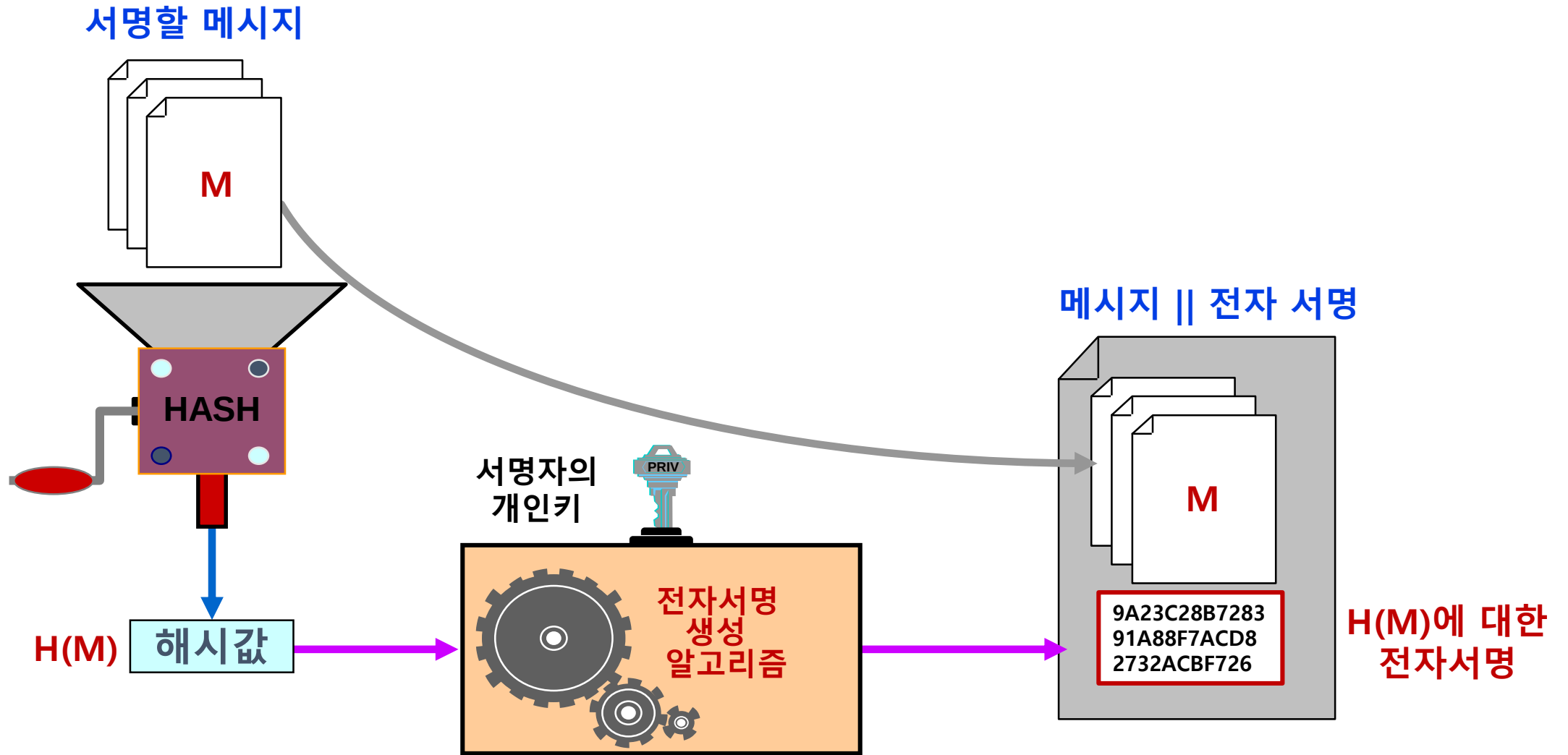
➤ 긴 문서의 서명에 대한 해결책

- 전체 메시지에 대해 서명을 생성하는 대신 메시지 다이제스트(해시 값)에 대해서만 서명 생성
- 안전성을 유지하면서 보다 서명 연산이 빠르게 수행됨
- 전자 서명의 실용성과 효율성 향상

➤ 해시 함수 이용

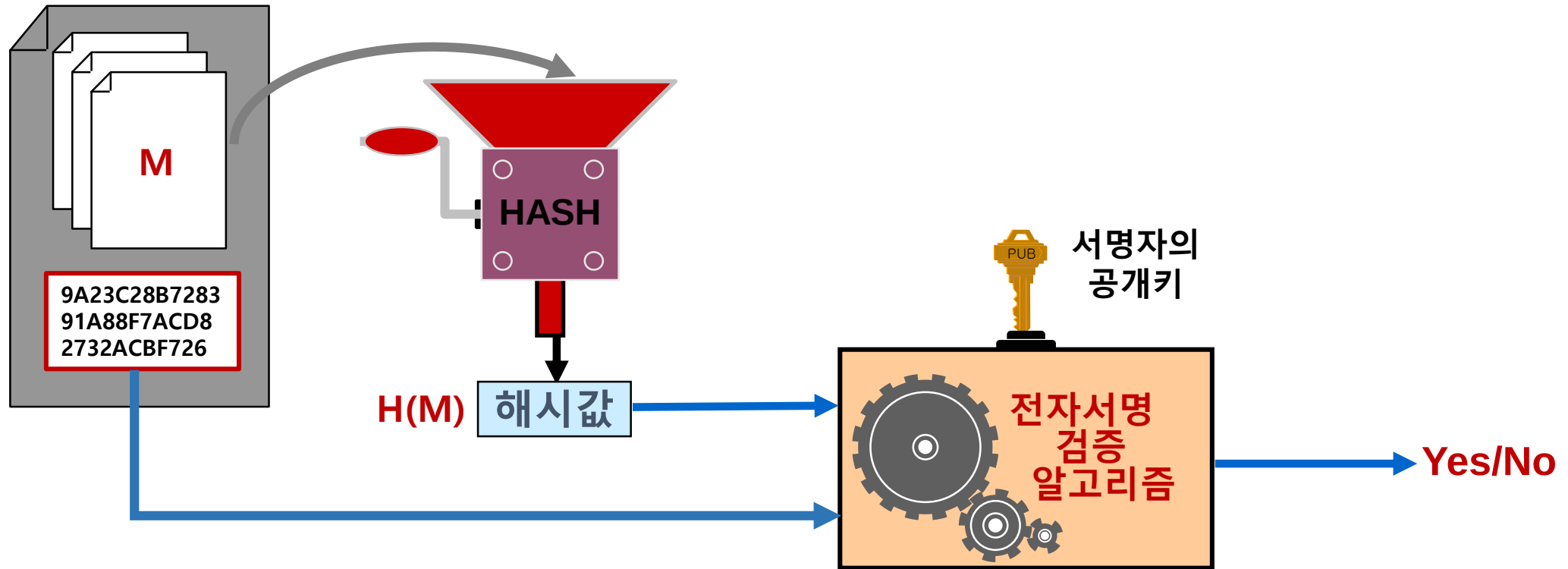
- 서명문의 무결성 보장
- 한번에 서명할 수 있는 크기로 압축

해시함수를 이용한 전자서명 생성



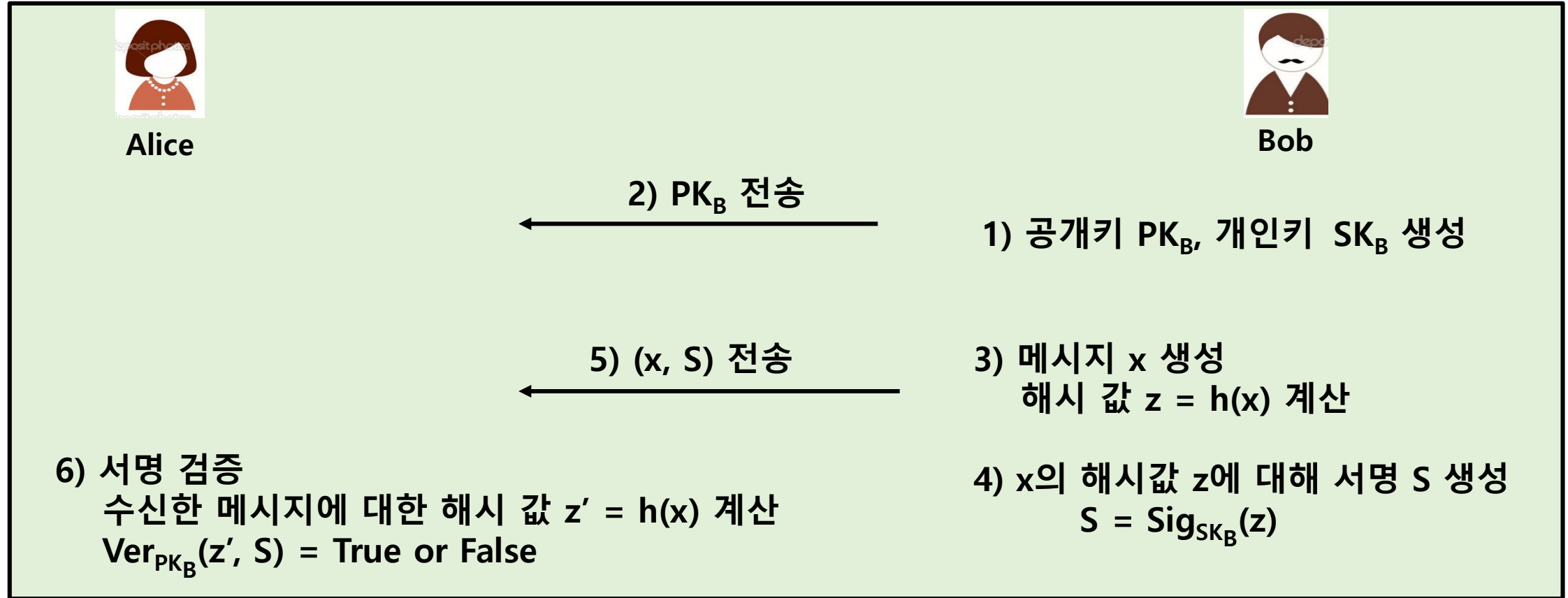
해시함수를 이용한 전자 서명 검증

메시지 || 전자 서명



※ 수신한 메시지의 해시 값을 서명 검증 알고리즘의 입력으로 사용

해시함수를 이용한 전자서명

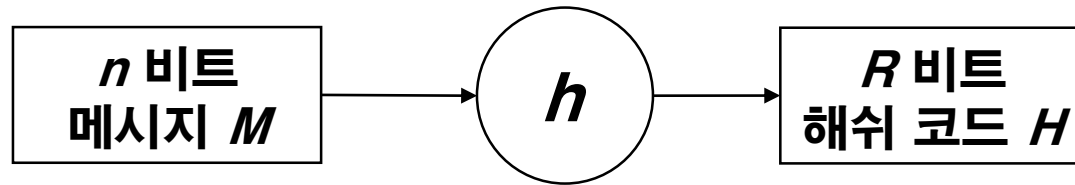


검증 결과가 true 인 경우, S 는 메시지 x 에 대한 Bob의 정당한 서명이며,
메시지 x 는 전송 중에 변경되지 않았음

해시 함수

➤ 정의

- n 비트의 긴 메시지 M 을 상대적으로 짧은 R 비트의 해시 코드(출력)로 변환하는 함수

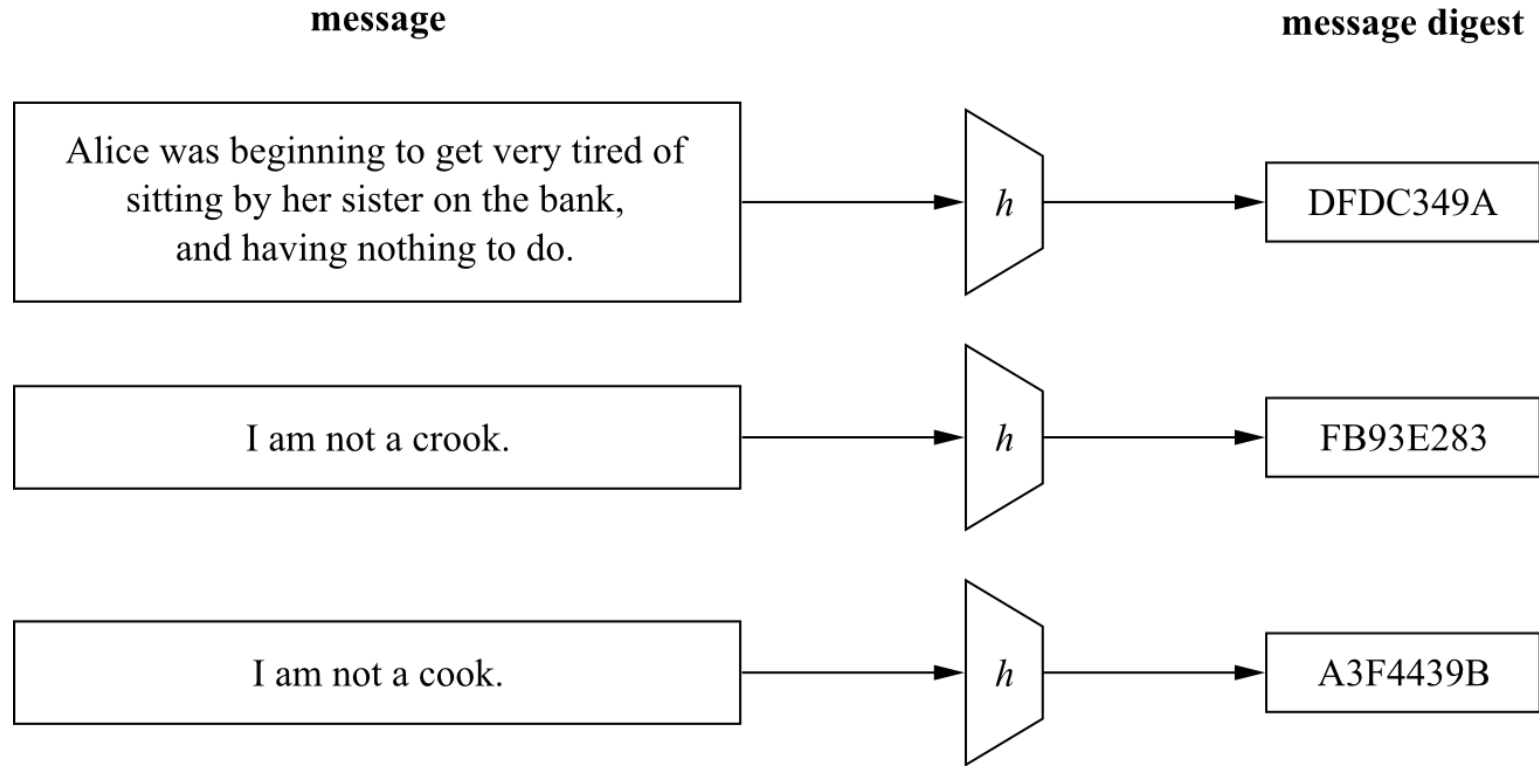


$$h : V^{\infty} \rightarrow V^R$$
$$h(M) = H \quad (n \gg R)$$

- 해시 함수의 연산은 **계산적으로 효율적**이어야 함

해시 함수

➤ 해시 함수의 입력-출력 동작 원리



- ※ 입력의 길이가 달라도 해시 함수의 출력은 항상 동일한 길이임
- ※ 입력이 약간 변경되더라도 출력은 완전히 다르게 나타남

암호학적 해시 함수의 안전성 요구사항

(1) 일방향성(onewayness, weak)

- 역상 찾기 어려움(preimage resistance)이라고도 함
- 해시 값 H 로부터 $h(M) = H$ 가 되는 M 을 찾는 것은 계산상 불가능

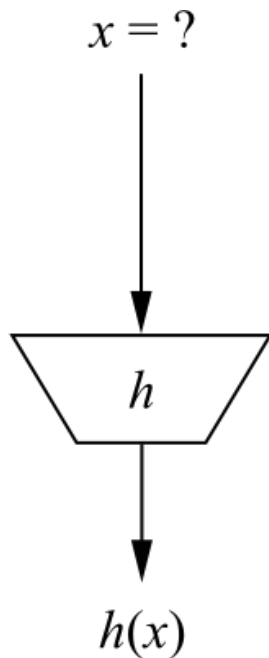
(2) 약한 충돌방지(Weak collision resistance)

- 두 번째 역상 찾기 어려움(second preimage resistance)이라고도 함
- M 과 해시 값 H 가 주어졌을 때 $h(M') = H$ 가 되는 $M \neq M'$ 인 M' 를 찾는 것이 계산상 불가능

(3) 강한 충돌 방지(Strong collision resistance)

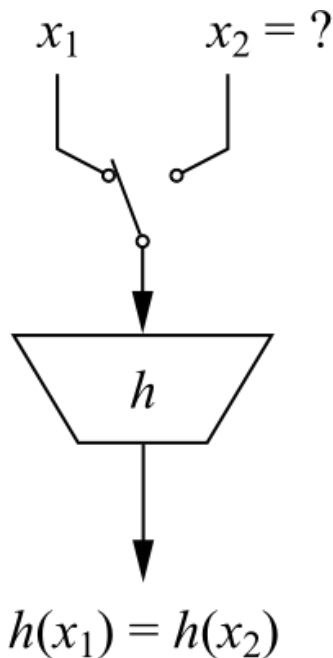
- $h(M) = h(M')$ 가 되는 서로 다른 M, M' 를 찾는 것이 계산상 불가능

암호학적 해시 함수의 안전성 요구사항



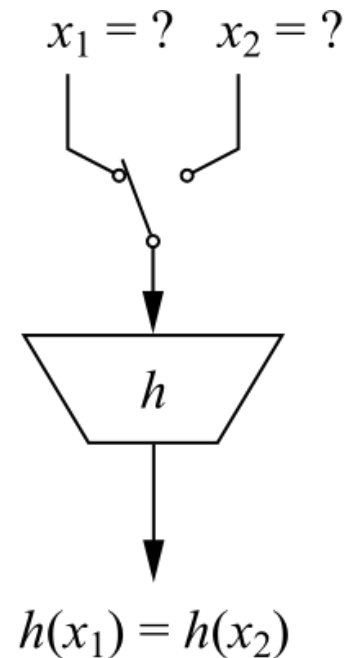
preimage resistance

(역상 찾기의 어려움)



second preimage
resistance

(두 번째 역상 찾기의 어려움)



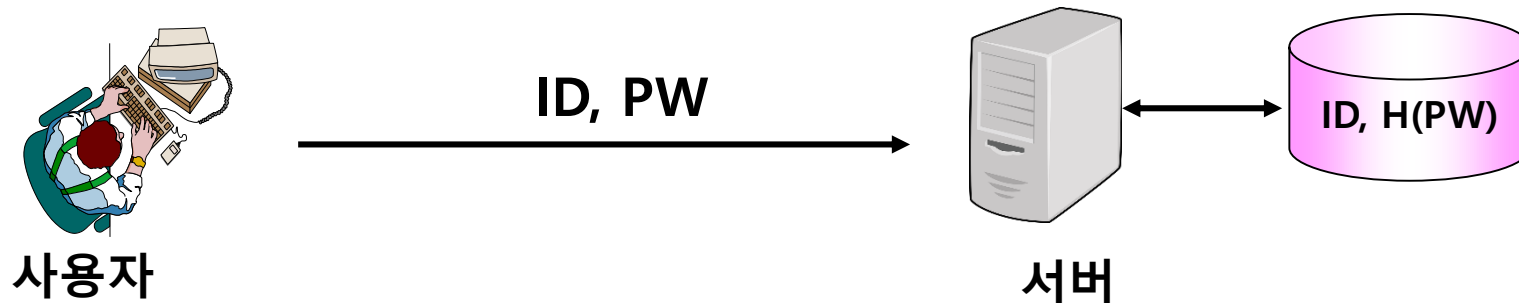
collision resistance

(충돌 방지)

암호학적 해시함수의 안전성 요구사항

(1) 일방향성(onewayness, weak)

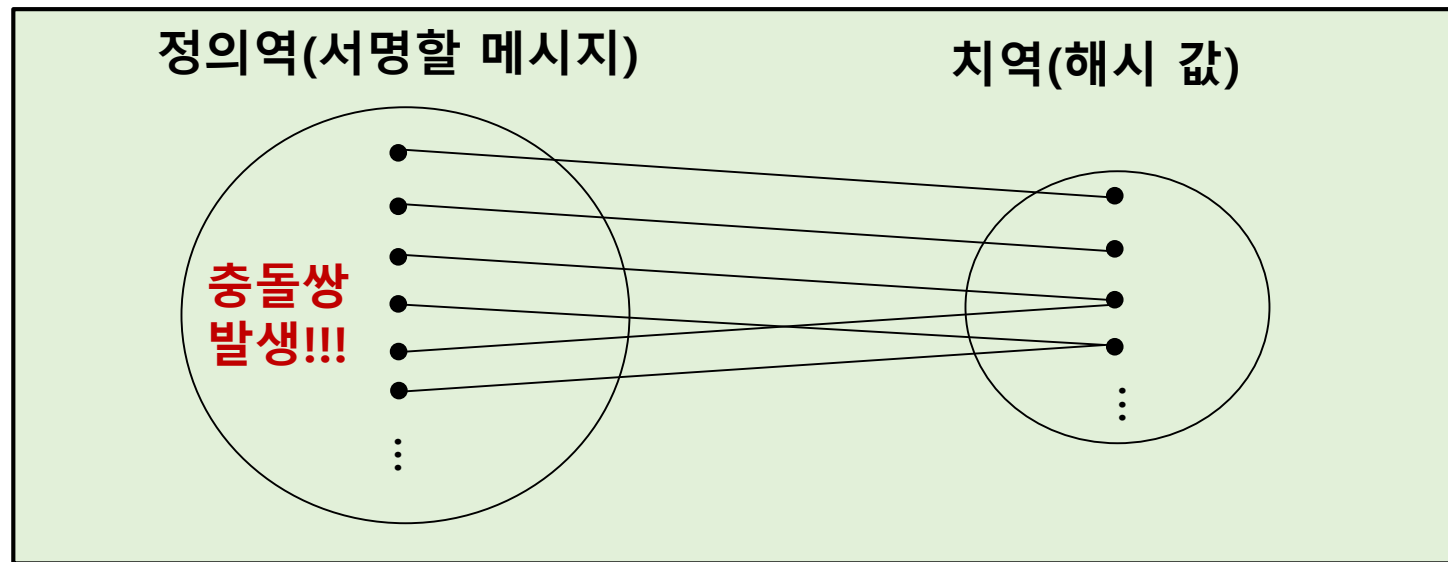
- 해시 값(출력)으로 부터 입력 메시지를 찾는 것은 계산상 불가능
- 해시 함수 사용 예
 - 서버에 사용자의 계정 정보(ID, PW)를 저장할 때, PW는 해시 값으로 저장
 - 패스워드 파일이 노출되더라도 실제 패스워드를 알아내는 것은 어렵게 함



암호학적 해시함수의 안전성 요구사항

(2) 약한 충돌 방지(Weak collision resistance)

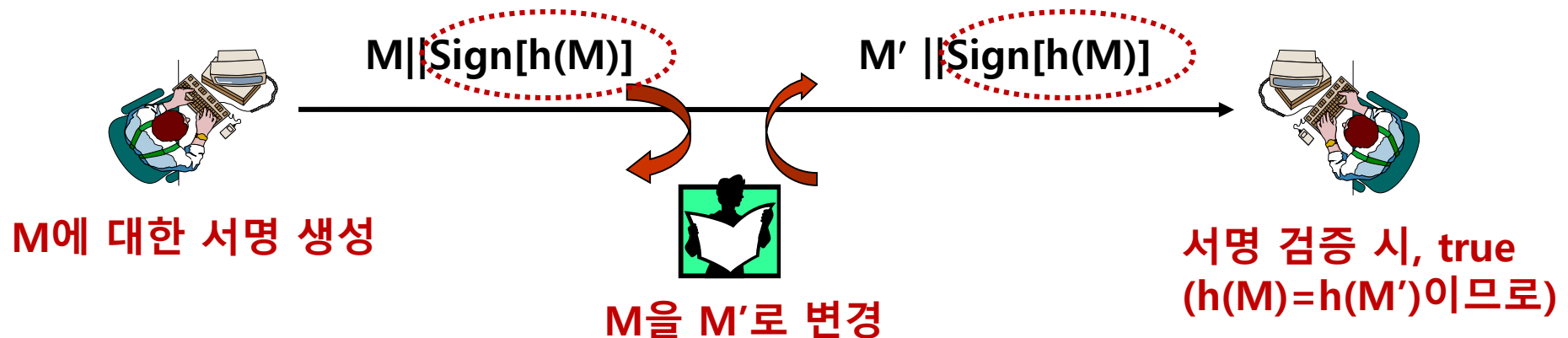
- 메시지 M 과 해시 값 H 가 주어졌을 때 $h(M') = H$ 이고 $M \neq M'$ 인 M' 를 찾는 것이 계산상 불가능해야 함
- 해시 함수는 긴 메시지에 대해 짧은 출력을 생성하므로, 정의역이 치역보다 크므로 충돌은 반드시 발생
- 그러나 충돌이 생기는 서로 다른 입력 M 과 M' 를 찾는 것이 계산상 불가능해야 함



암호학적 해시함수의 안전성 요구사항

➤ 약한 충돌방지 성질을 만족하지 않는 경우

- 메시지 M 과 해시 값 $H=h(M)$ 이 주어졌을 때, 동일한 해시값을 갖는 새로운 메시지 M' ($H=h(M')$ 인 M')를 찾는 것이 가능
- 두 메시지 M, M' 의 해시 값이 동일하면($h(M) = h(M')$), 메시지 M 에 대한 서명문과 메시지 M' 에 대한 서명이 동일
- 따라서, 메시지 M 에 대한 서명을 M' 의 서명으로 **재사용 가능**



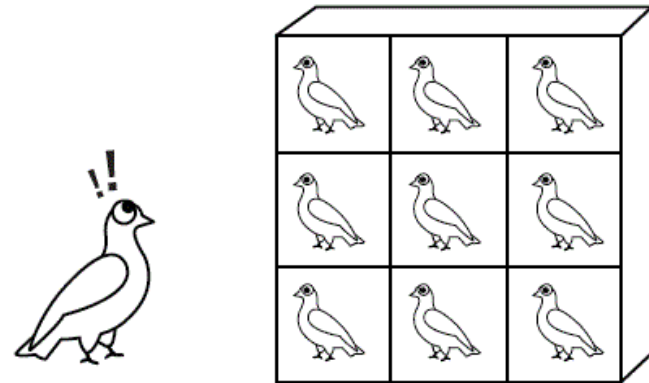
암호학적 해시함수의 안전성 요구사항

(3) 강한 충돌방지

- $h(M) = h(M')$ 가 되는 서로 다른 M, M' 를 찾는 것이 계산상 불가능해야 함
- 강한 충돌방지 성질을 만족하지 않으면, 같은 해시 값을 갖는 두 메시지를 찾을 수 있음
- 동일한 해시 값을 갖는 두 메시지 M, M' 를 찾으면, M 의 서명을 M' 의 서명으로 재사용 가능

암호학적 해시함수의 안전성 요구사항

- 해시 함수의 특성 상 동일한 해시 값을 갖는 충돌쌍은 반드시 존재
- 충돌이 생기는 서로 다른 M 과 M' 를 찾는 것이 계산상 불가능 하려면, **해시 함수의 출력 길이가 어느 정도 되어야 할까?**
- [참고] 비둘기 집의 원리(pigeonhole principle)
 - $(n+1)$ 마리의 비둘기가 n 개의 집에 들어가야 한다면, 최소한 한 개의 비둘기 집에는 두 마리 이상의 비둘기가 들어가야 한다는 원리



[참고] 생일 역설

➤ 생일 역설(Birthday paradox)

- 그룹 내에서 **최소 두 사람의 생일이 같을 확률이 50% 이상** 되려면 **몇 명의 사람이 모여야 할까?**

- $P(\text{2명의 생일이 같지 않을 확률}) = 1 - \frac{1}{365}$

- $P(\text{3명의 생일이 같지 않을 확률}) = (1 - \frac{1}{365})(1 - \frac{2}{365})$

- $P(\text{t명의 생일이 같지 않을 확률})$

$$= (1 - \frac{1}{365})(1 - \frac{2}{365}) \dots (1 - \frac{t-1}{365})$$

[참고] 생일 역설

➤ 그룹 내에서 최소 두 사람의 생일이 같을 확률이 50% 이상이 되려면 얼마나 많은 사람이 필요할까?

- $P(\text{최소 한번의 충돌이 있을 확률})$
 $= 1 - P(\text{충돌이 없을 확률})$

$$= 1 - (1 - \frac{1}{365}) \dots (1 - \frac{t-1}{365})$$

- 확률이 50%가 되는 t 값을 계산해보니, $t=23$ 일 때 확률 50.7%가 됨
- 즉, **23명만 모이면 생일이 같은 사람이 있을 확률이 50%가 넘게 됨**

[참고] 생일 역설

➤ 생일 역설(Birthday Paradox)

- n 개의 원소를 갖는 집합에서 임의로 하나를 선택한 후, 다시 집어넣고 다시 하나를 선택하는 것을 반복할 때 중복되는 원소가 선택될 확률
- n 개의 결과 값을 가지는 경우, 두 결과 값이 같을 확률이 0.5이상이 되게 할 최소의 개수는 $1.17\sqrt{n}$
- 해시 함수의 출력이 n 비트(출력 개수는 2^n)인 경우, 서로 다른 메시지에 대해 같은 해시 값이 나올 확률이 0.5이상이 되려면, 약 $\sqrt{2^n}$ 개의 해시 값을 계산하면 됨

해시함수의 안전성 요구사항

[충돌 쌍을 찾기 위해 필요한 해시 값의 개수]

충돌 쌍을 찾을 확률	해시 결과 길이				
	128비트	160비트	256비트	384비트	512비트
0.5	2^{65}	2^{81}	2^{129}	2^{193}	2^{257}
0.9	2^{67}	2^{82}	2^{130}	2^{194}	2^{258}

- 해시 함수를 비밀번호 저장에 사용하는 경우에는 일방향성만을 만족하면 되며, 상대적으로 짧은 출력을 갖는 해시 함수 사용가능
- 해시 함수를 전자서명에 사용하는 경우에는 **충돌방지가 매우 중요하며, 256비트 이상의 출력을 갖는 해시 함수를 사용해야 함**

해시 함수

➤ 해시 함수를 구축하는 방식

- **전용 해시함수** : 해시 함수로 동작하도록 특별히 설계된 알고리즘
- **블록 암호 기반의 해시 함수** : 해시 함수를 구성하기 위해 AES와 같은 블록 암호 알고리즘 이용

➤ 해시 함수는 임의의 길이의 메시지를 처리하여 고정된 출력 생성

➤ 입력을 동일한 크기의 여러 블록으로 분할하여 블록들을 차례로 해시 함수에 적용

➤ 해시 함수의 핵심기능은 **압축(Message digest)**

전용 해시 함수

➤ MD4

- R. Rivest가 개발한 해시 함수(1990년)
- 모든 연산이 AND, OR, XOR, NOT 같은 부울(Boolean) 연산으로 구성되어 매우 효율적
- MD5, SHA, RIPEMD 등의 알고리즘이 MD4에 기반

➤ MD5

- R. Rivest가 1991년 제안
- 보안 프로토콜에서 파일의 체크섬을 계산하거나 패스워드 해시값을 저장하는데 매우 널리 사용됨
- 임의의 길이의 입력에 대해 128비트의 해시 값 출력
- **충돌쌍 탐색에 대한 취약점 발견**
- 최근에 사용을 권고하지 않음

전용 해시 함수

➤ SHA(Secure Hash Algorithm)

- 미국 NIST에서 안전한 해시 함수로 SHA 개발
- 1993년 FIPS PUB 180으로 공표
- 1995년에 개정된 SHA-1(FIPS PUB 180-1)
- MD4에 기반하여 설계
- 2^{64} 미만의 길이를 갖는 입력에 대해 **160 비트의 해시 값 출력**
- 512 비트 단위로 동작
- SHA-2라 불리는 **SHA-224, SHA-256, SHA-384, SHA-512**가 발표됨 -> 2002년 FIPS PUB 180-2
- 2012년 10월 Keccak 알고리즘이 SHA-3로 최종 선정
- 2015년 SHA-3 발표

[MD4 계열의 해시 함수]

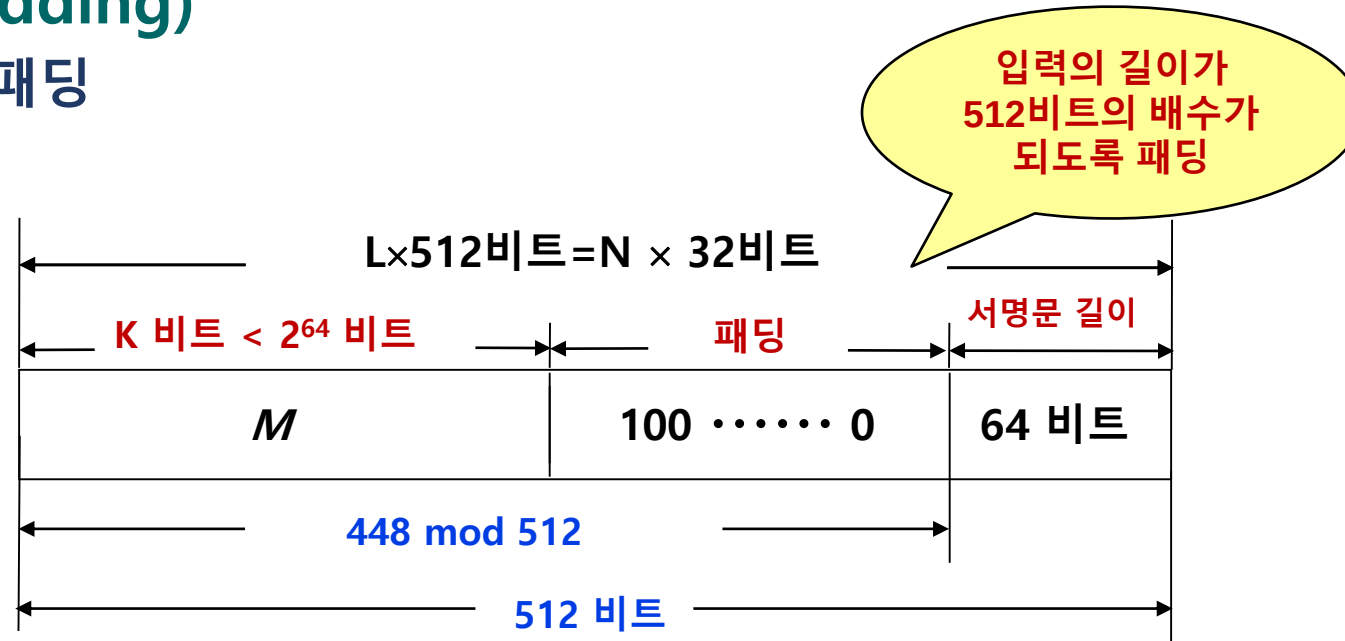
알고리즘		결과(비트)	입력(비트)	라운드 수	충돌 탐색 여부
MD5		128	512	64	O
SHA-1		160	512	80	X
SHA-2	SHA-224	224	512	64	X
	SHA-256	256	512	64	X
	SHA-384	384	1024	80	X
	SHA-512	512	1024	80	X

SHA-1

- 길이 최대 2^{64} 비트를 갖는 메시지에 대해 160비트의 해시 값 출력
- 해시를 하기 전에 메시지의 길이가 512의 배수가 되도록 패딩 수행
- 압축 함수는 20개의 라운드를 갖는 4개의 함수로 구성

➤ 선처리 : 패딩(padding)

▪ 입력 메시지 M의 패딩



- 패딩 패턴 : **100.....0**
- 마지막 64비트 : 패딩 전 메시지의 길이 표시
- 길이(64비트)까지 합쳐서 512비트의 배수가 되어야 하므로, 마지막 블록이 448비트가 되도록 패딩 수행

SHA-1

➤ [예제] $M = abc$

01100001 01100010 01100011
a(97) b(98) c(99)

➤ 메시지의 길이가 24비트이므로 $448 \bmod 512$ 가 되도록 424 비트 패딩하고, 마지막 64비트 길이를 포함시킴

▪ 패딩 값 : 100000...0
423 bits

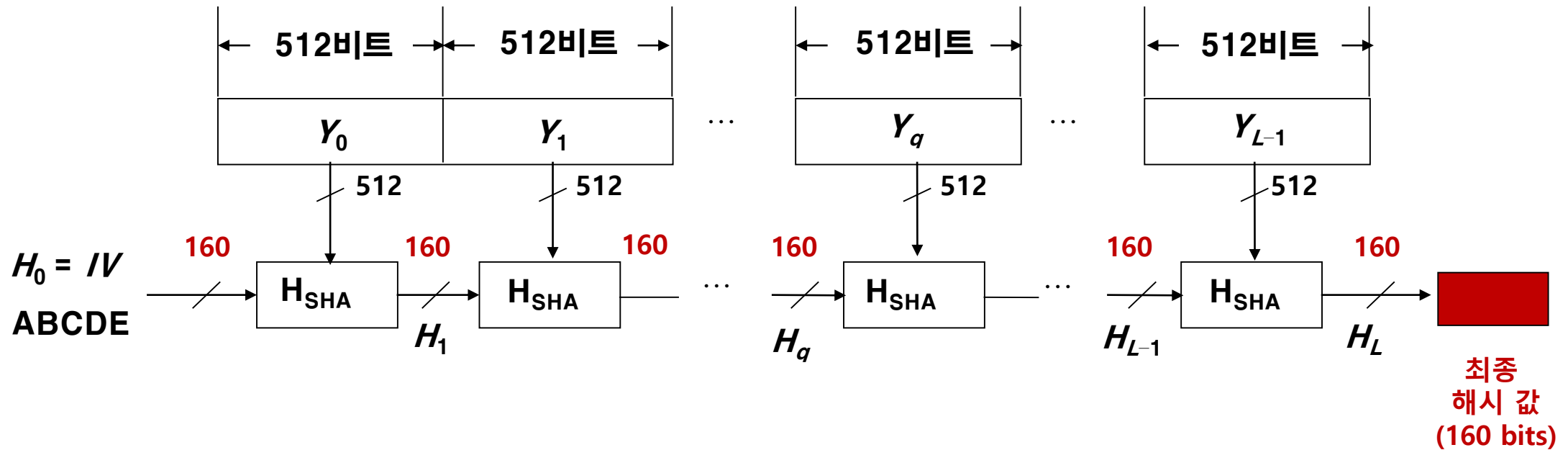
길이 : 00000000...0011000
64 bits

➤ 패딩 결과

01100001 01100010 01100011 1000...0 00000000...00011000
423 bits 64 bits
448 bits
512 bits

SHA-1

➤ 해시 계산 과정



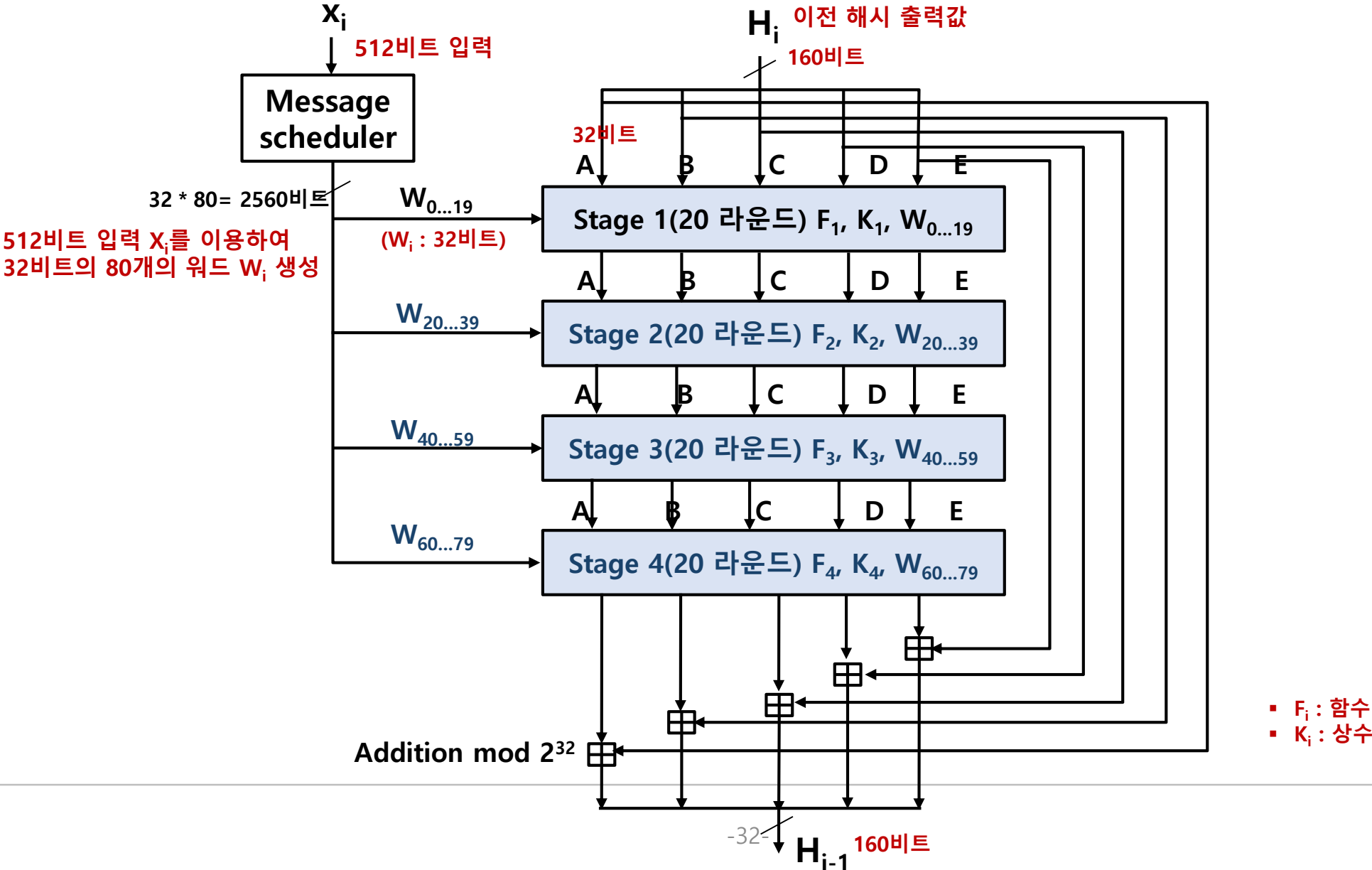
➤ 초기값 H_0

- 해시 함수의 중간 값과 최종 결과값을 저장하기 위해 5개의 32비트 레지스터 이용

➤ 레지스터 초기 값(32비트)

- $A = H_0^{(0)} = 6\ 7\ 4\ 5\ 2\ 3\ 0\ 1$
- $B = H_0^{(1)} = E\ F\ C\ D\ A\ B\ 8\ 9$
- $C = H_0^{(2)} = 9\ 8\ B\ A\ D\ C\ F\ E$
- $D = H_0^{(3)} = 1\ 0\ 3\ 2\ 5\ 4\ 7\ 6$
- $E = H_0^{(4)} = C\ 3\ D\ 2\ E\ 1\ F\ 0$

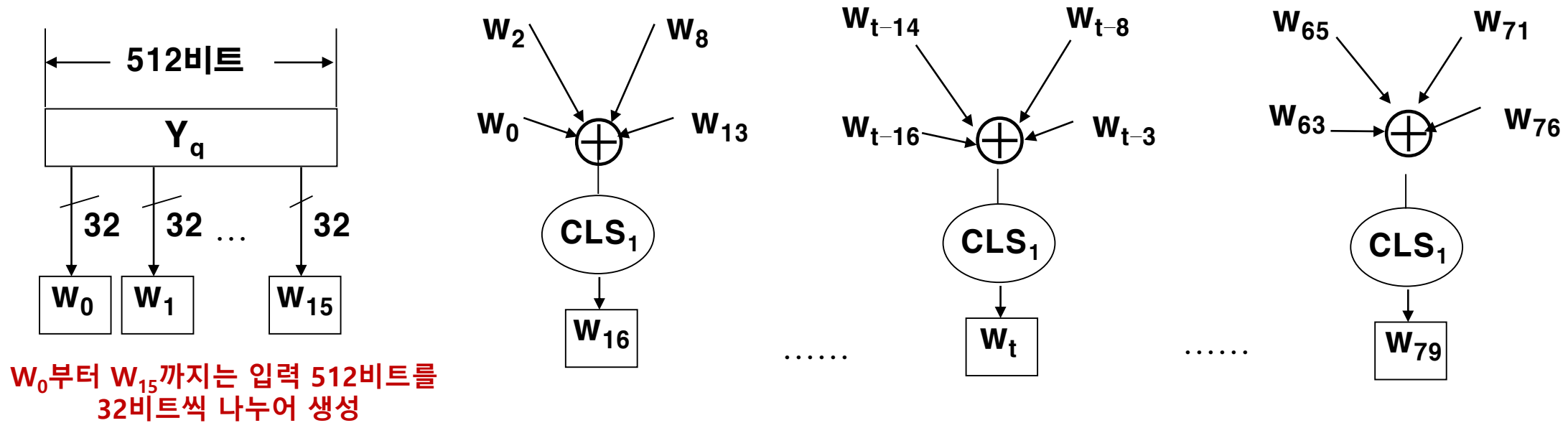
SHA-1



SHA-1

➤ SHA-1의 메시지 스케줄러

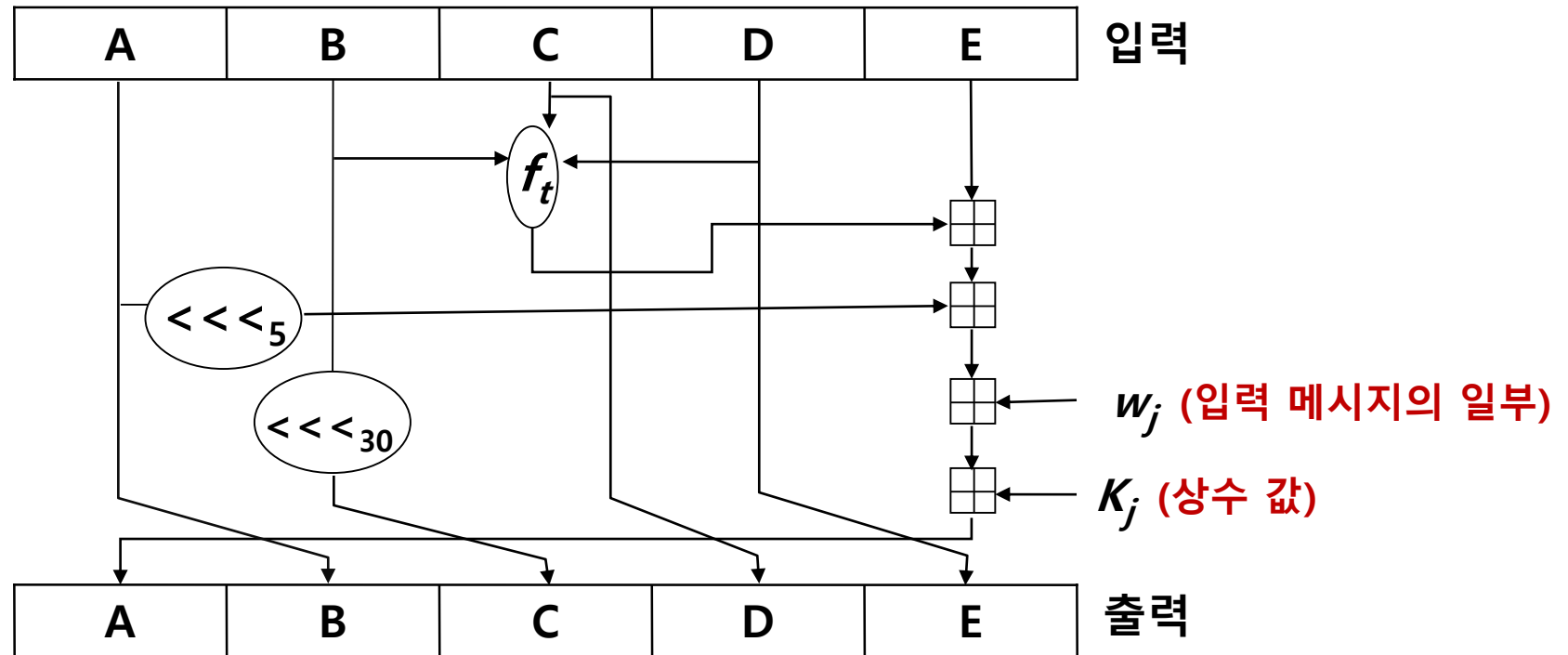
- 512비트 입력 X_i 를 이용하여 32비트의 80개의 워드 W_i 생성



$$W_t = CLS_1(W_{t-16} \oplus W_{t-14} \oplus W_{t-8} \oplus W_{t-3}) \quad (16 \leq t \leq 79)$$

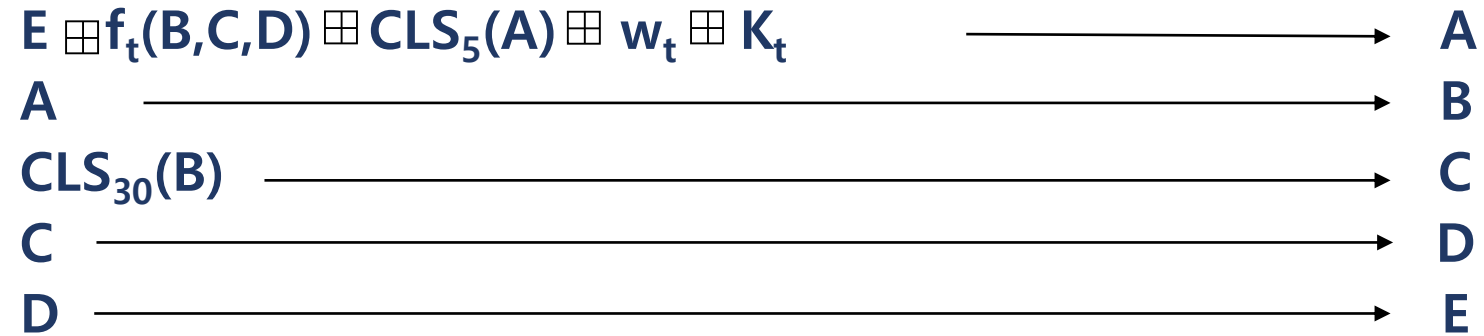
SHA-1

- 4개의 stage는 비슷한 구조를 가지지만, 서로 다른 내부함수 f_t 와 상수 K_t 사용
- 각 stage는 20개의 라운드로 구성



[SHA-1의 단계 t 의 라운드 j]

SHA-1



- t : 라운드 수 ($0 \leq t \leq 79$)
- f_t : 기약 논리함수
- CLS_s : s 비트 순환 좌측 시프트
- w_t : 메시지 스케줄러에서 생성한 입력 32 비트
- K_t : 상수

SHA-1

➤ SHA-1의 상수 K_t 값

stage	라운드	상수 K_t
1	0 ~ 19	5 A 8 2 7 9 9 9
2	20 ~ 39	6 E D 9 E B A 1
3	40 ~ 59	8 F 1 B B C D C
4	60 ~ 79	C A 6 2 C 1 D 6

SHA-1

➤ SHA-1의 함수 f_t

stage	라운드	함수 f_t
1	0 ~ 19	$f_1(B, C, D) = (B \wedge C) \vee (\bar{B} \wedge D)$
2	20 ~ 39	$f_2(B, C, D) = B \oplus C \oplus D$
3	40 ~ 59	$f_3(B, C, D) = (B \wedge C) \vee (B \wedge D) \vee (C \wedge D)$
4	60 ~ 79	$f_4(B, C, D) = B \oplus C \oplus D$

SHA-1(예제)

➤ 메시지 $M = abc$ 에 대한 SHA-1의 첫 번째 라운드 결과를 구하시오.

1. M 을 비트열로 나타내기

- $a = 97 = 0x61 = 01100001$
- $b = 98 = 0x62 = 01100010$
- $c = 99 = 0x63 = 01100011$

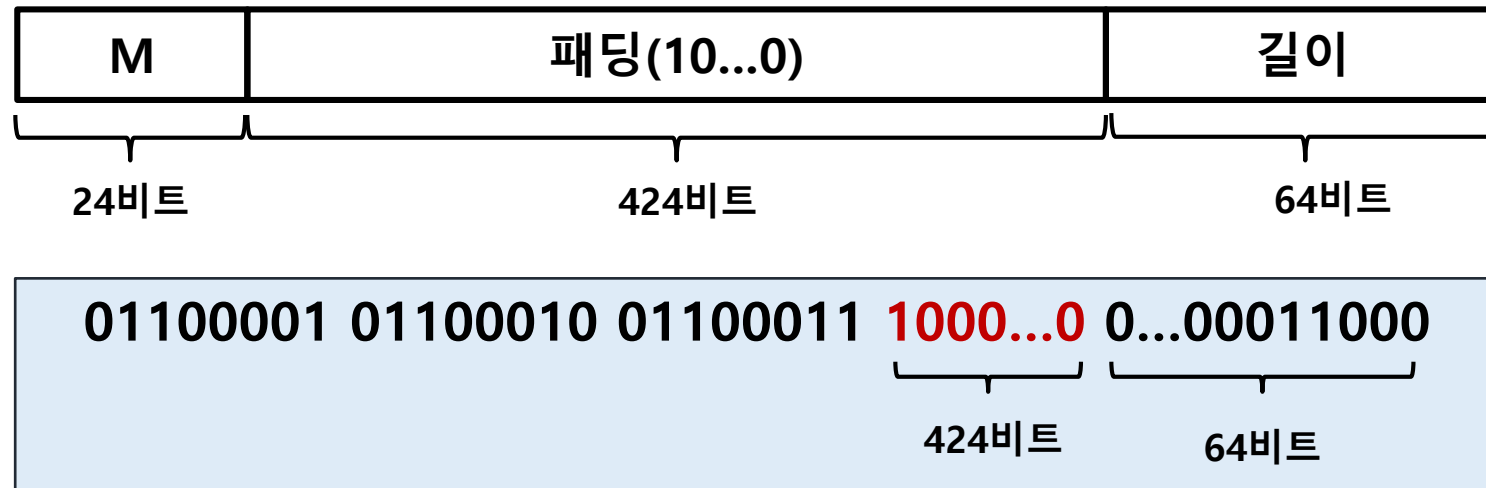
$M = 01100001\ 01100010\ 01100011$

SHA-1(예제)

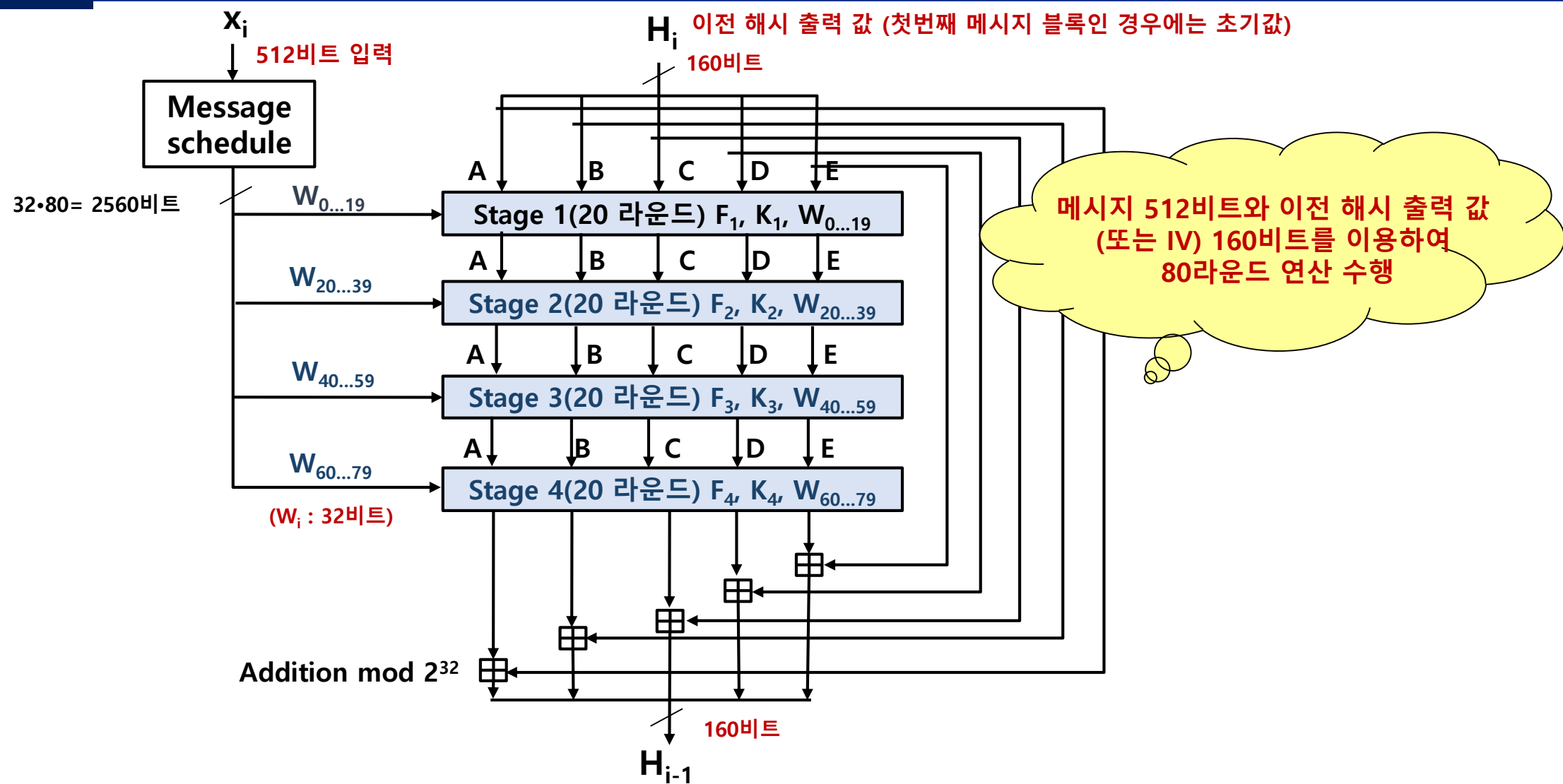
2. 패딩 하기

M = 01100001 01100010 01100011(24비트)

- 64비트의 길이를 포함해서 512비트가 되도록 패딩



SHA-1(예제)



SHA-1(예제)

3. 입력 메시지 W_0 구하기

- 512비트 입력 메시지를 32비트씩 16개 워드로 나눔

01100001 01100010 01100011 1000...0 00000000 00000000011000

$$W_0 = 61626380$$

$$W_1 = 00000000$$

$$W_2 = 00000000$$

$$W_3 = 00000000$$

$$W_4 = 00000000$$

$$W_5 = 00000000$$

$$W_6 = 00000000$$

$$W_7 = 00000000$$

$$W_8 = 00000000$$

$$W_9 = 00000000$$

$$W_{10} = 00000000$$

$$W_{11} = 00000000$$

$$W_{12} = 00000000$$

$$W_{13} = 00000000$$

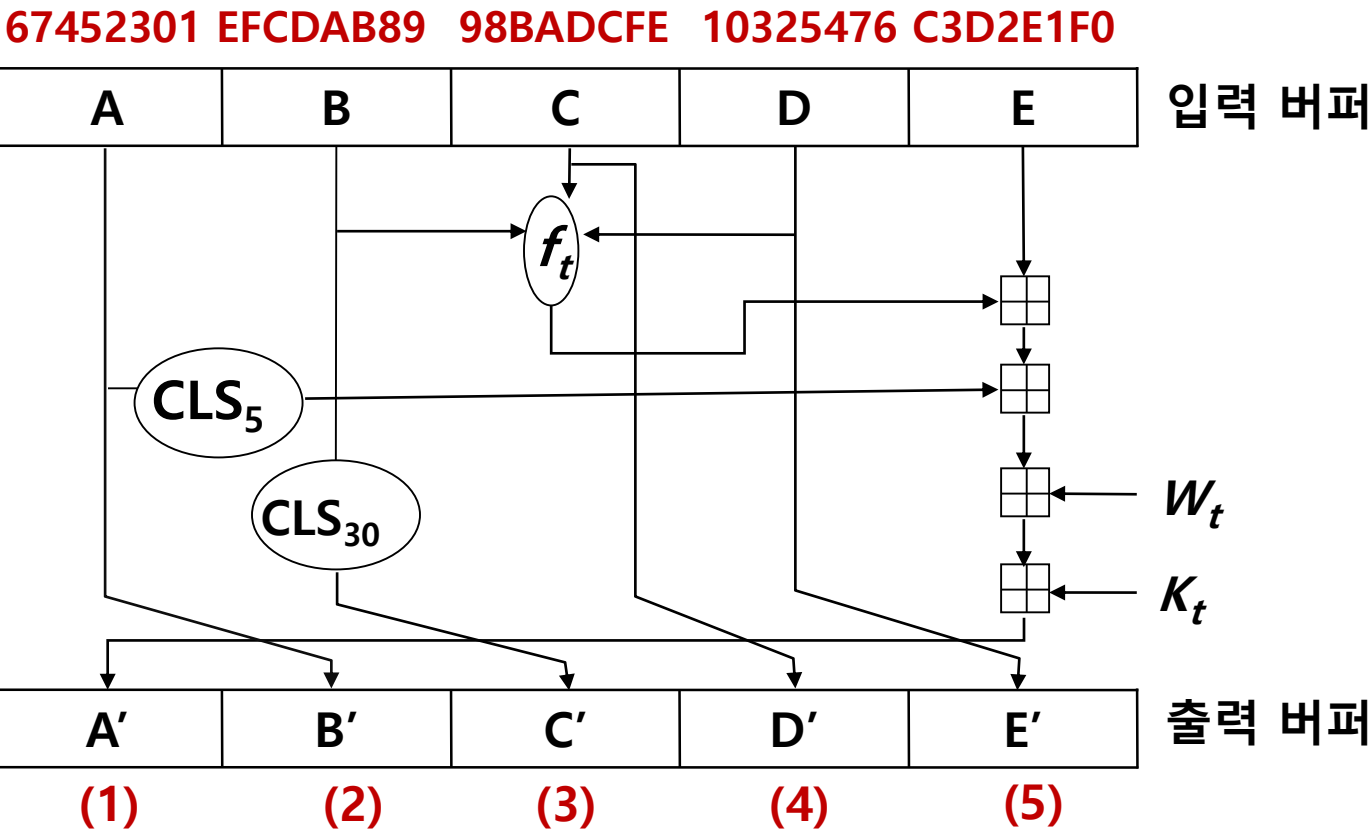
$$W_{14} = 00000000$$

$$W_{15} = 00000018$$

$$W_0 = 0x61626380 = 0110\ 0001\ 0110\ 0010\ 0110\ 0011\ 1000\ 0000$$

SHA-1(예제)

4. 라운드 함수 수행하기



SHA-1(예제)

➤ 레지스터 초기 값

- $A = H_0^{(0)} = 6\ 7\ 4\ 5\ 2\ 3\ 0\ 1$
- $B = H_0^{(1)} = E\ F\ C\ D\ A\ B\ 8\ 9$
- $C = H_0^{(2)} = 9\ 8\ B\ A\ D\ C\ F\ E$
- $D = H_0^{(3)} = 1\ 0\ 3\ 2\ 5\ 4\ 7\ 6$
- $E = H_0^{(4)} = C\ 3\ D\ 2\ E\ 1\ F\ 0$

SHA-1(예제)

$$(1) A' = E \boxplus f_1(B,C,D) \boxplus \text{CLS}_5(A) \boxplus w_0 \boxplus K_0$$

① $f_1(B,C,D)$

② $E \boxplus f_1(B,C,D)$

③ ② $\boxplus \text{CLS}_5(A)$

④ ③ $\boxplus w_0$

⑤ ④ $\boxplus K_0$

▪ $f_1(B, C, D) = (B \wedge C) \vee (\overline{B} \wedge D)$

▪ $W_0 = 61626380$

▪ $K_0 = 5A827999$

SHA-1(예제)

① $f_1(B,C,D)$

$$\begin{aligned} f_t(B,C,D) &= (B \wedge C) \vee (\overline{B} \wedge D) \\ &= (E F C D A B 8 9 \wedge 9 8 B A D C F E) \vee \\ &\quad (1 0 3 2 5 4 7 6 \wedge 1 0 3 2 5 4 7 6) \end{aligned}$$

1110	1111	1100	1101	1010	1011	1000	1001
$\wedge$	1001	1000	1011	1010	1101	1100	1111
1000	1000	1000	1000	1000	1000	1000	1000

0001	0000	0011	0010	0101	0100	0111	0110
$\wedge$	0001	0000	0011	0010	0101	0100	0111
0001	0000	0011	0010	0101	0100	0111	0110

1000	1000	1000	1000	1000	1000	1000	1000
$\vee$	0001	0000	0011	0010	0101	0100	0111
1001	1000	1011	1010	1101	1100	1111	1110

SHA-1(예제)

② $E \boxplus f_1(B,C,D)$

$E = C\ 3\ D\ 2\ E\ 1\ F\ 0$

$f_1(B,C,D) = 1001\ 1000\ 1011\ 1010\ 1101\ 1100\ 1111\ 1110$

	1100	0011	1101	0010	1110	0001	1111	0000
$\boxplus$	1001	1000	1011	1010	1101	1100	1111	1110
	0101	1100	1000	1101	1011	1110	1110	1110

SHA-1(예제)

③ ② $\boxplus$ CLS₅(A)

A = 67452301

0110 0111 0100 0101 0010 0011 0000 0001

왼쪽으로 5번 cyclic shift 수행

1110 1000 1010 0100 0110 0000 0010 1100

	0101	1100	1000	1101	1011	1110	1110	1110
$\boxplus$	1110	1000	1010	0100	0110	0000	0010	1100
	0100	0101	0011	0010	0001	1111	0001	1010

SHA-1(예제)

④ ③ $\boxplus$ W_0

$W_0 = 61626380$

$= 0110\ 0001\ 0110\ 0010\ 0110\ 0011\ 1000\ 0000$

	0100	0101	0011	0010	0001	1111	0001	1010
$\boxplus$	0110	0001	0110	0010	0110	0011	1000	0000
	1010	0110	1001	0100	1000	0010	1001	1010

SHA-1(예제)

⑤ ④ $\boxplus$ K_0

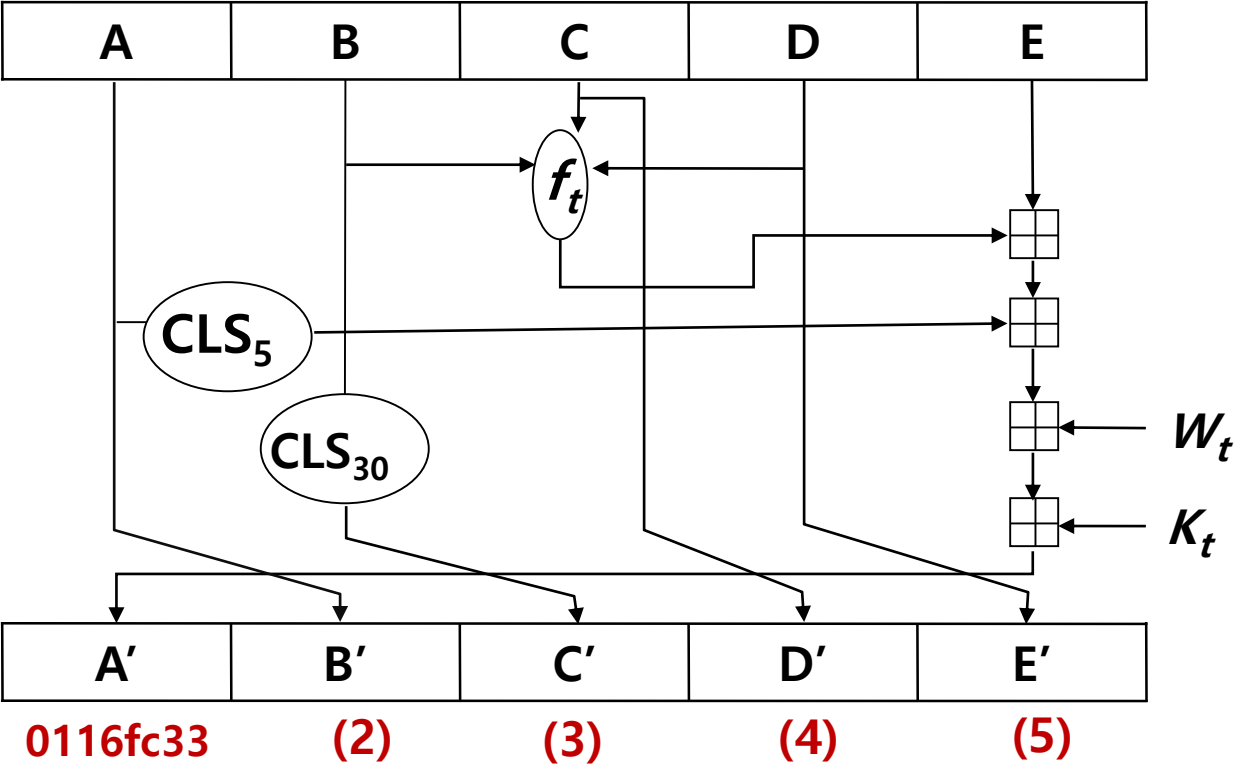
$$\begin{aligned} K_0 &= 5 \text{ A } 8 \text{ 2 } 7 \text{ 9 } 9 \text{ 9} \\ &= 0101 \text{ 1010 } 1000 \text{ 0010 } 0111 \text{ 1001 } 1001 \text{ 1001} \end{aligned}$$

	1010	0110	1001	0100	1000	0010	1001	1010
$\boxplus$	0101	1010	1000	0010	0111	1001	1001	1001
	0000	0001	0001	0110	1111	1100	0011	0011

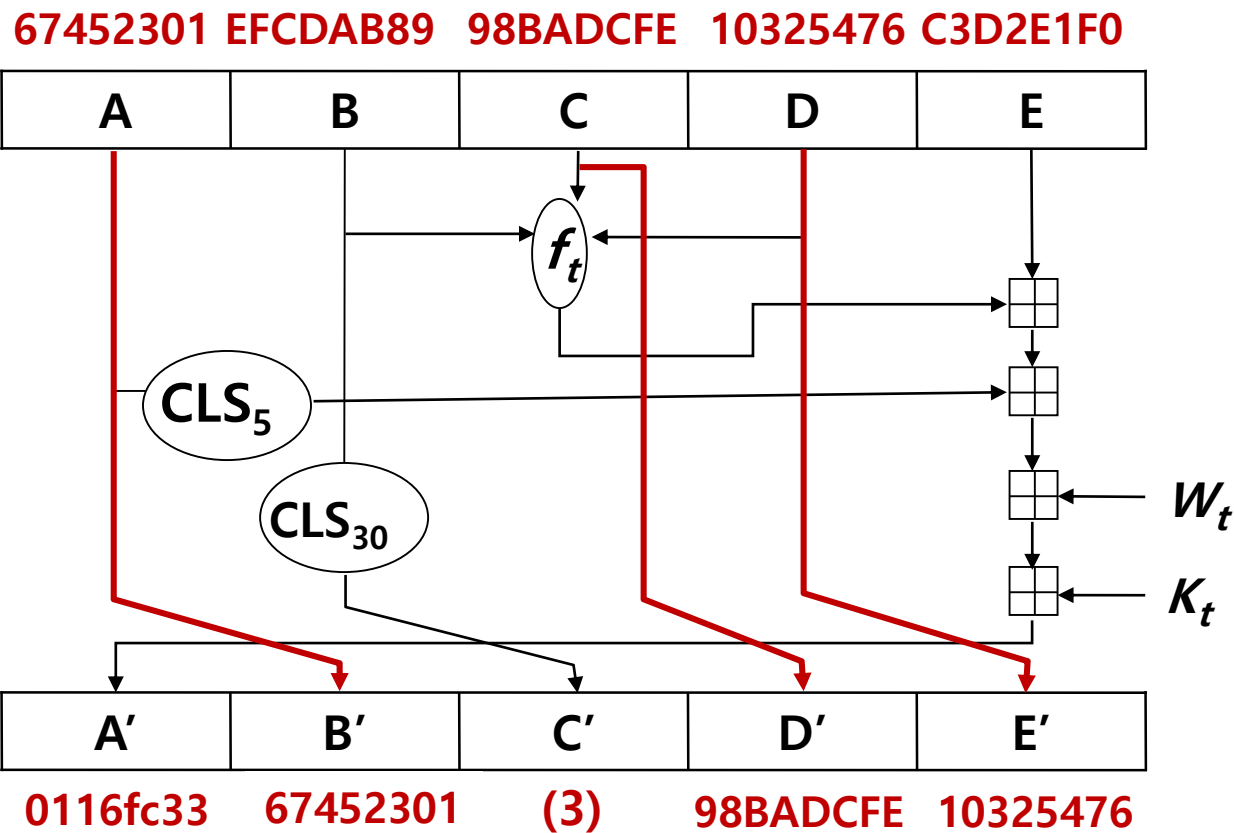
$$\begin{aligned} (1) \ A' &= 0000 \text{ 0001 } 0001 \text{ 0110 } 1111 \text{ 1100 } 0011 \text{ 0011} \\ &\rightarrow \mathbf{0 \ 1 \ 1 \ 6 \ f \ c \ 3 \ 3} \end{aligned}$$

SHA-1(예제)

67452301 EFC DAB89 98BADC FE 10325476 C3D2E1F0



SHA-1(예제)



SHA-1(예제)

$$(3) C' = \text{CLS}_{30}(B)$$

$$B = \text{EFCDAB89}$$

$$= 1110\ 1111\ 1100\ 1101\ 1010\ 1011\ 1000\ 1001$$

30비트 왼쪽으로 cyclic shift 수행

1110 1111 1100 1101 1010 1011 1000 1001

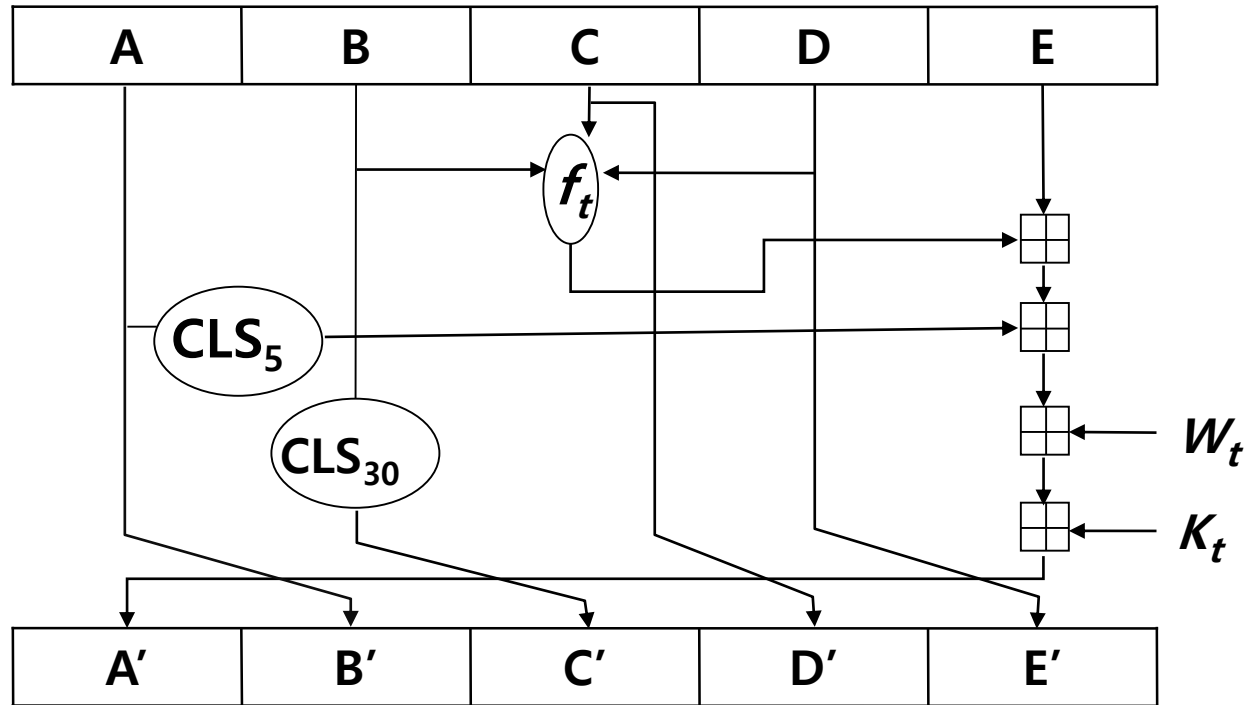
-> 0111 1011 1111 0011 0110 1010 1110 0010

$$C' = 7bf36ae2$$

SHA-1(예제)

➤ 첫번째 라운드 수행 후 A, B, C, D, E의 값

67452301 EFC DAB89 98BADC FE 10325476 C3D2E1F0



0116fc33 67452301 7bf36ae2 98BADC FE 10325476

SHA-1(예제)

➤ 80라운드 반복

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>
<i>t</i> = 0 :	0116fc33	67452301	7bf36ae2	98badcfe	10325476
<i>t</i> = 1 :	8990536d	0116fc33	59d148c0	7bf36ae2	98badcfe
<i>t</i> = 2 :	a1390f08	8990536d	c045bf0c	59d148c0	7bf36ae2
<i>t</i> = 3 :	cdd8e11b	a1390f08	626414db	c045bf0c	59d148c0
<i>t</i> = 4 :	cf499de	cdd8e11b	284e43c2	626414db	c045bf0c
<i>t</i> = 5 :	3fc7ca40	cf499de	f3763846	284e43c2	626414db
<i>t</i> = 6 :	993e30c1	3fc7ca40	b3f52677	f3763846	284e43c2
<i>t</i> = 7 :	9e8c07d4	993e30c1	0ff1f290	b3f52677	f3763846
<i>t</i> = 8 :	4b6ae328	9e8c07d4	664f8c30	0ff1f290	b3f52677
<i>t</i> = 9 :	8351f929	4b6ae328	27a301f5	664f8c30	0ff1f290
<i>t</i> = 10 :	fbda9e89	8351f929	12dab8ca	27a301f5	664f8c30
<i>t</i> = 11 :	63188fe4	fbda9e89	60d47e4a	12dab8ca	27a301f5
<i>t</i> = 12 :	4607b664	63188fe4	7ef6a7a2	60d47e4a	12dab8ca
<i>t</i> = 13 :	9128f695	4607b664	18c623f9	7ef6a7a2	60d47e4a
<i>t</i> = 14 :	196bee77	9128f695	1181ed99	18c623f9	7ef6a7a2

라운드마다 입력 메시지 W_i 와 $F_t()$ 함수, 상수 K_t 를 변경하여 80번의 라운드 수행

SHA-1(예제)

$t = 15 :$	20bdd62f	196bee77	644a3da5	1181ed99	18c623f9
$t = 16 :$	4e925823	20bdd62f	c65afb9d	644a3da5	1181ed99
$t = 17 :$	82aa6728	4e925823	c82f758b	c65afb9d	644a3da5
$t = 18 :$	dc64901d	82aa6728	d3a49608	c82f758b	c65afb9d
$t = 19 :$	fd9e1d7d	dc64901d	20aa99ca	d3a49608	c82f758b
$t = 20 :$	1a37b0ca	fd9e1d7d	77192407	20aa99ca	d3a49608
$t = 21 :$	33a23bfc	1a37b0ca	7f67875f	77192407	20aa99ca
$t = 22 :$	21283486	33a23bfc	868dec32	7f67875f	77192407
$t = 23 :$	d541f12d	21283486	0ce88eff	868dec32	7f67875f
$t = 24 :$	c7567dc6	d541f12d	884a0d21	0ce88eff	868dec32
$t = 25 :$	48413ba4	c7567dc6	75507c4b	884a0d21	0ce88eff
$t = 26 :$	be35fbd5	48413ba4	b1d59f71	75507c4b	884a0d21
$t = 27 :$	4aa84d97	be35fbd5	12104ee9	b1d59f71	75507c4b
$t = 28 :$	8370b52e	4aa84d97	6f8d7ef5	12104ee9	b1d59f71
$t = 29 :$	c5fbaf5d	8370b52e	d2aa1365	6f8d7ef5	12104ee9
$t = 30 :$	1267b407	c5fbaf5d	a0dc2d4b	d2aa1365	6f8d7ef5
$t = 31 :$	3b845d33	1267b407	717eebd7	a0dc2d4b	d2aa1365
$t = 32 :$	046faa0a	3b845d33	c499ed01	717eebd7	a0dc2d4b
$t = 33 :$	2c0ebc11	046faa0a	cee1174c	c499ed01	717eebd7
$t = 34 :$	21796ad4	2c0ebc11	811bea82	cee1174c	c499ed01
$t = 35 :$	dcbbb0cb	21796ad4	4b03af04	811bea82	cee1174c
$t = 36 :$	0f511fd8	dcbbb0cb	085e5ab5	4b03af04	811bea82
$t = 37 :$	dc63973f	0f511fd8	f72eec32	085e5ab5	4b03af04
$t = 38 :$	4c986405	dc63973f	03d447f6	f72eec32	085e5ab5
$t = 39 :$	32de1cba	4c986405	f718e5cf	03d447f6	f72eec32
$t = 40 :$	fc87dedf	32de1cba	53261901	f718e5cf	03d447f6

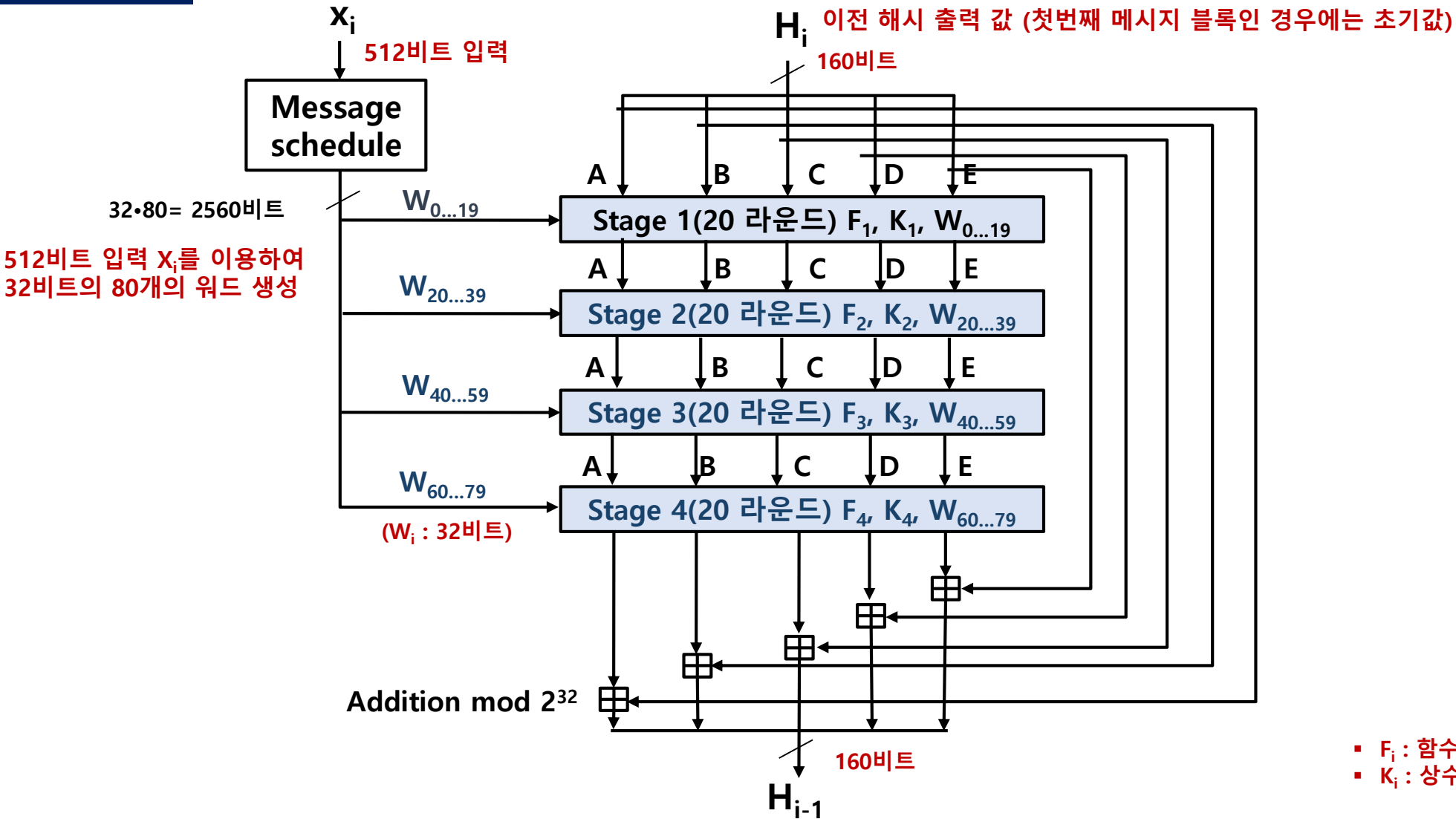
SHA-1(예제)

$t = 41 :$	970a0d5c	fc87dedf	8cb7872e	53261901	f718e5cf
$t = 42 :$	7f193dc5	970a0d5c	ff21f7b7	8cb7872e	53261901
$t = 43 :$	ee1b1aaf	7f193dc5	25c28357	ff21f7b7	8cb7872e
$t = 44 :$	40f28e09	ee1b1aaf	5fc64f71	25c28357	ff21f7b7
$t = 45 :$	1c51e1f2	40f28e09	fb86c6ab	5fc64f71	25c28357
$t = 46 :$	a01b846c	1c51e1f2	503ca382	fb86c6ab	5fc64f71
$t = 47 :$	bead02ca	a01b846c	8714787c	503ca382	fb86c6ab
$t = 48 :$	baf39337	bead02ca	2806e11b	8714787c	503ca382
$t = 49 :$	120731c5	baf39337	afab40b2	2806e11b	8714787c
$t = 50 :$	641db2ce	120731c5	eebce4cd	afab40b2	2806e11b
$t = 51 :$	3847ad66	641db2ce	4481cc71	eebce4cd	afab40b2
$t = 52 :$	e490436d	3847ad66	99076cb3	4481cc71	eebce4cd
$t = 53 :$	27e9f1d8	e490436d	8e11eb59	99076cb3	4481cc71
$t = 54 :$	7b71f76d	27e9f1d8	792410db	8e11eb59	99076cb3
$t = 55 :$	5e6456af	7b71f76d	09fa7c76	792410db	8e11eb59
$t = 56 :$	c846093f	5e6456af	5edc7ddb	09fa7c76	792410db
$t = 57 :$	d262ff50	c846093f	d79915ab	5edc7ddb	09fa7c76
$t = 58 :$	09d785fd	d262ff50	f211824f	d79915ab	5edc7ddb

SHA-1(예제)

$t = 59 :$	3f52de5a	09d785fd	3498bfd4	f211824f	d79915ab
$t = 60 :$	d756c147	3f52de5a	4275e17f	3498bfd4	f211824f
$t = 61 :$	548c9cb2	d756c147	8fd4b796	4275e17f	3498bfd4
$t = 62 :$	b66c020b	548c9cb2	f5d5b051	8fd4b796	4275e17f
$t = 63 :$	6b61c9e1	b66c020b	9523272c	f5d5b051	8fd4b796
$t = 64 :$	19dfa7ac	6b61c9e1	ed9b0082	9523272c	f5d5b051
$t = 65 :$	101655f9	19dfa7ac	5ad87278	ed9b0082	9523272c
$t = 66 :$	0c3df2b4	101655f9	0677e9eb	5ad87278	ed9b0082
$t = 67 :$	78dd4d2b	0c3df2b4	4405957e	0677e9eb	5ad87278
$t = 68 :$	497093c0	78dd4d2b	030f7cad	4405957e	0677e9eb
$t = 69 :$	3f2588c2	497093c0	de37534a	030f7cad	4405957e
$t = 70 :$	c199f8c7	3f2588c2	125c24f0	de37534a	030f7cad
$t = 71 :$	39859de7	c199f8c7	8fc96230	125c24f0	de37534a
$t = 72 :$	edb42de4	39859de7	f0667e31	8fc96230	125c24f0
$t = 73 :$	11793f6f	edb42de4	ce616779	f0667e31	8fc96230
$t = 74 :$	5ee76897	11793f6f	3b6d0b79	ce616779	f0667e31
$t = 75 :$	63f7dab7	5ee76897	c45e4fdb	3b6d0b79	ce616779
$t = 76 :$	a079b7d9	63f7dab7	d7b9da25	c45e4fdb	3b6d0b79
$t = 77 :$	860d21cc	a079b7d9	d8fdf6ad	d7b9da25	c45e4fdb
$t = 78 :$	5738d5e1	860d21cc	681e6df6	d8fdf6ad	d7b9da25
$t = 79 :$	42541b35	5738d5e1	21834873	681e6df6	d8fdf6ad

SHA-1(예제)



SHA-1(예제)

- 80 라운드 수행 후 레지스터의 초기 입력 값과 80 라운드 결과에 대해 덧셈 수행

$$A = 67452301 + 42541b35 = a9993e36$$

$$B = efcdab89 + 5738d5e1 = 4706816a$$

$$C = 98badcfe + 21834873 = ba3e2571$$

$$D = 10325476 + 681e6df6 = 7850c26c$$

$$E = c3d2e1f0 + d8fdf6ad = 9cd0d89d$$

- M=abc에 대한 160비트 해시 값

a9993e36 4706816a ba3e2571 7850c26c 9cd0d89d

SHA-2

[SHA-1, SHA-2 알고리즘]

Algorithm	Message size(bits)	Block size (bits)	Word size (bits)	Message Digest size(bits)
SHA-1	$<2^{64}$	512	32	160
SHA-224	$<2^{64}$	512	32	224
SHA-256	$<2^{64}$	512	32	256
SHA-384	$<2^{128}$	1024	64	384
SHA-512	$<2^{128}$	1024	64	512

- SHA-224, 256, 384, 512를 SHA-2라 함(FIPS 180-3)
- SHA-3 : 2012년 10월 Keccak 최종 선정

5.6 SHA-3

➤ SHA-3

- SHA-1 알고리즘과 SHA-2 알고리즘 제품군을 대체하기 위해 설계된 미국 정부 표준 알고리즘
- SHA-3는 keccak으로 알려진 이전에 개발된 기술에 그 기반을 두고 있음
- 이전에 개발된 대부분의 암호화 해시 함수는 MD5와 SHA-1을 포함하여 머클-담가드 (Merkle-Damgard) 구성에 기반을 두고 있음
 - 약점: 길이 확장 공격(length extension attack)
- SHA-3 알고리즘은 메시지를 블록으로 해시('스폰지' 기술 사용)

5.7 HMAC

- 메시지 무결성을 검증하기 위해 메시지 인증 코드(MAC)를 이용
 - 대칭키 암호 알고리즘의 CBC 모드 이용
 - MAC을 계산하기 위해 해시 함수를 사용할 수 있음(Hashed MAC)
- 앨리스가 $h(M)$ 을 계산하여 M 과 $h(M)$ 을 밥에게 보냄으로써 M 의 무결성을 보장할 수 있는가?



5.7 HMAC

- 해시 함수의 연산에는 키를 사용하지 않으므로, 메시지의 해시 값만을 추가하는 것은 무결성을 보장할 수 없음
- 해시 함수의 연산에 **키를 추가**하는 방식 필요

- M : 송신할 메시지
- $h()$: 해시 함수
- K_{AB} : Alice와 Bob 사이의 비밀 정보(키)



Alice

(1) 전송할 메시지에 대한
메시지 인증코드 계산
 $MAC = h(M || K_{AB})$

(2) $M || MAC$ 전송



Bob

(3) $MAC' = h(M || K_{AB})$ 계산
 $MAC \stackrel{?}{=} MAC'$ 인지 확인

5.8 해시함수 응용 분야

➤ 온라인 입찰

- 엘리스, 밥, 척이 입찰하고자 하는 경매 물건이 온라인에 나왔다고 가정
 - 엘리스는 10달러에 입찰을 하고 밥은 12달러에 입찰을 한다고 가정 → 만약 척이 두 사람의 입찰 금액을 입찰 마감 전에 미리 안다면 12달러 1센트를 적어내서 입찰에 성공
 - ↳ 나중에 입찰하는 것이 유리할 수 있기 때문에 어느 누구도 입찰 금액을 먼저 적어내려고 하지 않음
 - 이러한 우려를 없애기 위해 각 입찰자는 자신의 입찰 금액을 결정(비밀)하고, **입찰 금액의 해시 값 제출**
 - 세 사람이 입찰을 제출하면 해시 값을 온라인에 게시해 모두가 볼 수 있도록 함
- 이 방법이 입찰을 직접 제출하는 것보다 나은 이유는 무엇일까?
 - 해시 값은 입찰 자체에 대한 정보는 노출시키지 않음
 - 해시 값을 한 번 제출하고 나면 입찰을 바꾸는 것이 불가능하므로, 입찰을 직접 제출했을 때 발생하는 문제를 예방할 수 있음

5.8 해시함수 응용 분야

- 메시지 인증 코드 생성
- 전자서명 생성을 위한 해시 값 생성
- 서버에 사용자 패스워드를 저장하는 경우
- 소프트웨어 변경 검출
- 디지털 포렌식에서 파일 무결성 확인
- 의사 난수 생성기(Pseudo Random Number Generator)
 - 해시 함수의 출력이 랜덤하므로 난수 생성에 이용
- 일회용 패스워드(One-Time Password) 생성
 - 현재 시간과 seed 값을 입력으로 해시 값 생성

정리

- 해시 함수는 임의의 길이의 메시지를 입력으로 상대적으로 짧은 길이의 고정된 출력을 생성하는 함수
- 해시 함수의 중요한 응용분야는 전자서명과 메시지 인증 코드(MAC) 생성
 - 해시함수는 원래 키를 사용하지 않으며, MAC을 생성하기 위해서는 키를 추가하는 방법 필요
- 암호학적 해시 함수는 일방향성, 약한 충돌 방지 및 강한 충돌 방지와 같은 보안 요구사항을 만족해야 함
- 충돌 쌍을 찾는 공격을 방지하기 위해 해시 함수의 결과는 최소 160 비트 길이이어야 하며, 256 비트 또는 그 이상은 장기간 보안에 적합
- 널리 사용되던 MD5는 안전하지 않고 SHA-1에 대한 심각한 보안 결함이 발견되었으므로 사용이 중단되어야 하며, SHA-2 알고리즘은 안전